Buổi 6

Các thành viên đặc biệt của lớp

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Đối tượng hằng và hàm thành viên const

1.1 Đối tượng hằng và hàm thành viên const
1.2 Thành viên dữ liệu const và tham số const

2. Lớp là thành viên của lớp khác (composition)

2.1 Quan hệ "có một" – đối tượng thành viên
2.2 Thứ tự gọi constructor và destructor với đối tượng thành viên

3. Hàm friend và lớp friend

3.1 Hàm friend
3.2 Đặc điểm và ứng dụng của friend; lớp friend

4. Con trỏ this

4.1 Con trỏ this
4.2 Ứng dụng của this: gọi hàm nối tiếp

5. Đối tượng cấp phát động và thành viên static

5.1 Đối tượng được cấp phát động
5.2 Thành viên static của lớp
5.3 Ví dụ static: mã tự tăng và lãi suất dùng chung

6. Tổng kết và thực hành

6.1 Tóm tắt buổi 6
6.2 Bài thực hành buổi 6 – Lớp NhanVien chứa Date, đếm đối tượng bằng static
6.3 Câu hỏi và bài tập
6.4 Bài tập dự đoán kết quả

1.1 Đối tượng hằng và hàm thành viên const

- **Nguyên lý đặc quyền tối thiểu:** cái gì không cần sửa thì khai báo là **không được sửa**.

Đối tượng hằng – hàm thành viên const

- **Đối tượng hằng:** `const Time Noon(12, 0, 0);` – không được thay đổi sau khi khởi tạo.
- **Hàm thành viên const:** thêm `const` **sau danh sách tham số** (cả ở prototype và định nghĩa) – cam kết **không sửa** thành viên dữ liệu: `int GetHour() const;`
- **Quy tắc:** đối tượng hằng **chỉ** được gọi các hàm thành viên **const** (constructor, destructor là ngoại lệ).

```
class Time
{  public:
      Time(int h = 0, int m = 0, int s = 0);
      void SetHour(int h);              // sửa ⇒ không const
      int  GetHour() const;             // chỉ đọc ⇒ const
      void PrintMilitary() const;
   private:
      int Hour, Minute, Second;
};
int Time::GetHour() const { return Hour; }   // const ở cả
định nghĩa
int main()
{  const Time Noon(12, 0, 0);
   Noon.PrintMilitary();                // 12:00 – hợp lệ
   cout << " " << Noon.GetHour();       // 12   – hợp lệ
}
```

Ví dụ 6.1 – Thông báo lỗi của g++

```
void Bad() const { H = 5; }
// error: assignment of member 'Time::H'
//        in read-only object
const Time t{};
t.Set(3);    // Set không const
// error: passing 'const Time' as 'this' argument
//        discards qualifiers
t.Get();     // Get chỉ đọc nhưng QUÊN const ⇒ lỗi tương tự
t.GetC();    // GetC() const ⇒ hợp lệ
```

- **Bài học:** hàm chỉ đọc mà quên `const` (như Get) thì **không dùng được** với đối tượng hằng, với tham số `const Time &` ⇒ hãy đánh dấu const cho **mọi** hàm không sửa dữ liệu.

1.2 Thành viên dữ liệu const và tham số const

- **Thành viên dữ liệu const:** giá trị cố định suốt đời đối tượng nhưng có thể **khác nhau** giữa các đối tượng ⇒ **bắt buộc** khởi tạo bằng **danh sách khởi tạo**:

```
class Increment
{  public:
      Increment(int c = 0, int i = 1)
         : Count(c), Step(i)      // BẮT BUỘC với Step
      {  }
      void AddIncrement() { Count += Step; }
      void Print() const
      {  cout << "Count = " << Count << ", Step = " << Step
<< endl;  }
   private:
      int Count;
      const int Step;             // thành viên const
};
Increment v(10, 5);
for (int j = 1; j <= 3; j++) { v.AddIncrement(); v.Print(); }
```

```
Count = 15, Step = 5
Count = 20, Step = 5
Count = 25, Step = 5
```

- Gán trong thân constructor `{ Step = i; }` ⇒ lỗi: **assignment of read-only member 'Increment::Step'** (const chỉ khởi tạo được, không gán được).
- **Tham số là đối tượng hằng:** truyền `const Time &` – không sao chép, không sửa; chỉ gọi được hàm const:

```
void InGio(const Time &t)
{  t.PrintMilitary();    // phải là hàm const
}
```

Vị trí của const	Ý nghĩa
`const Time t;`	Đối tượng hằng
`int GetHour() const;`	Hàm không sửa đối tượng
`const int Step;`	Thành viên dữ liệu hằng
`void F(const Time &t);`	Hàm không sửa đối số
`const int &GetRef() const;`	Trả về tham chiếu chỉ đọc (khắc phục BadSetHour buổi 5)

2.1 Quan hệ "có một" – đối tượng thành viên

- **Composition (hợp thành):** một lớp có thành viên dữ liệu là **đối tượng của lớp khác** – quan hệ "**có một**" (has-a). Ví dụ: nhân viên **có** ngày sinh, ngày vào làm; xe **có** động cơ.

```
class Date
{  public:
      Date(int d = 1, int m = 1, int y = 1900);
      ~Date();
      void Print() const;
   private:
      int Day, Month, Year;
};
class Employee
{  public:
      Employee(const char *fname, const char *lname,
               int bd, int bm, int by,     // ngày sinh
               int hd, int hm, int hy);    // ngày vào làm
      ~Employee();
      void Print() const;
   private:
      char FirstName[25], LastName[25];
      const Date BirthDate;      // đối tượng thành viên
      const Date HireDate;
};
```

- **Khởi tạo đối tượng thành viên** bằng **danh sách khởi tạo** – truyền tham số cho constructor của Date:

```
Employee::Employee(const char *fname, const char *lname,
                   int bd, int bm, int by,
                   int hd, int hm, int hy)
   : BirthDate(bd, bm, by), HireDate(hd, hm, hy)
{  strcpy(FirstName, fname);
   strcpy(LastName, lname);
   cout << "Employee constructor: "
        << FirstName << " " << LastName << endl;
}
void Employee::Print() const
{  cout << LastName << ", " << FirstName << "\nHired: ";
   HireDate.Print();                // gọi hàm của Date
   cout << "  Birth date: ";
   BirthDate.Print();
   cout << endl;
}
```

- Nếu không có danh sách khởi tạo, Date phải có **constructor mặc định**; BirthDate là `const` ⇒ **bắt buộc** khởi tạo trong danh sách.

Ví dụ 6.2 – Date in thông báo trong constructor/destructor

```
int main()
{  Employee e("Bob", "Jones",
              24, 7, 1949, 12, 3, 1988);
   cout << "\n";
   e.Print();
   cout << "\n";
}
```

- **Quy tắc:**
  - **Tạo:** đối tượng thành viên được tạo **trước**, theo **thứ tự khai báo** trong lớp (BirthDate rồi HireDate), sau đó **mới** đến thân constructor của lớp chứa.
  - **Hủy: ngược lại**: destructor lớp chứa chạy trước, rồi hủy các thành viên theo thứ tự ngược khai báo.
- Giống xây nhà: đặt **móng** (thành viên) trước rồi mới xây **nhà**; phá thì dỡ **nhà** trước rồi mới phá **móng**.

```
Date constructor: 24/7/1949
Date constructor: 12/3/1988
Employee constructor: Bob Jones

Jones, Bob
Hired: 12/3/1988  Birth date: 24/7/1949

Employee destructor: Jones, Bob
Date destructor: 12/3/1988
Date destructor: 24/7/1949
```

- Đổi thứ tự trong danh sách khởi tạo thành `: HireDate(…), BirthDate(…)` **không** thay đổi thứ tự tạo (g++ còn cảnh báo `-Wreorder`).

friend

Hàm **friend** của một lớp là hàm **không phải thành viên** của lớp nhưng được quyền truy cập các thành viên **private** và **protected** của lớp. Khai báo bằng từ khóa `friend` **bên trong** lớp. Tương tự, `friend class B;` cho mọi hàm thành viên của lớp B quyền truy cập lớp hiện tại.

```
class Count
{  friend void SetX(Count &, int);   // khai báo friend
   public:
      Count() { X = 0; }
      void Print() const { cout << X << endl; }
   private:
      int X;
};
void SetX(Count &c, int val)         // KHÔNG có Count::
{  c.X = val;    // hợp lệ vì SetX là friend
}
int main()
{  Count c;
   cout << "c.X after instantiation: ";  c.Print();
   SetX(c, 8);
   cout << "c.X after call to SetX: ";   c.Print();
}
```

```
c.X after instantiation: 0
c.X after call to SetX: 8
```

- SetX **không** phải hàm thành viên (định nghĩa không có `Count::`) nhưng gán được `c.X` private.
- Bỏ dòng `friend` ⇒ lỗi: **'int Count::X' is private within this context**.

3.2 Đặc điểm và ứng dụng của friend; lớp friend

- **Đặc điểm:**
  - Quan hệ friend do **lớp cấp** (lớp chủ động khai báo ai là bạn), không phải hàm tự nhận.
  - Không phải thành viên ⇒ gọi như hàm thường `SetX(c, 8)`, **không** có con trỏ this; đặt ở vùng public/private đều như nhau.
  - Friend **không** đối xứng (A là bạn của B không suy ra B là bạn của A), **không** bắc cầu, **không** kế thừa.
- **Khi nào dùng?**
  - Hàm cần truy cập private của **hai lớp** cùng lúc (ví dụ nhân Matrix với Vector).
  - Đa năng hóa toán tử mà toán hạng trái không phải đối tượng của lớp: `cout << a`, `2 * a` (buổi 7, 8).
- **Lưu ý:** friend **làm yếu** tính đóng gói ⇒ chỉ dùng khi thật cần.

- **Lớp friend:** `friend class NganHang;` ⇒ **mọi** hàm thành viên của NganHang truy cập được private của TaiKhoan.

```
class TaiKhoan
{  friend class NganHang;        // lớp bạn
   public:
      TaiKhoan(double sd = 0);
      double GetSoDu() const { return SoDu; }
   private:
      double SoDu;
};
class NganHang
{  public:
      void ChuyenTien(TaiKhoan &a, TaiKhoan &b,
                      double t)
      {  if (t > 0 && t <= a.SoDu)
         {  a.SoDu -= t;  b.SoDu += t;  }
      }
};
TaiKhoan x(1000), y(500);
NganHang nh;
nh.ChuyenTien(x, y, 300);
cout << x.GetSoDu() << " " << y.GetSoDu();  // 700 800
```

- Thiết kế này cho phép **chỉ** ngân hàng chuyển tiền trực tiếp; các lớp khác vẫn phải qua giao diện public.

Con trỏ this

Mỗi hàm thành viên (không static) nhận một tham số **ẩn** là con trỏ **this** trỏ tới **đối tượng đang gọi hàm**. Khi viết `Hour` trong hàm thành viên, trình biên dịch hiểu là `this->Hour`.

```
class Test
{  public:
      Test(int a = 0) { X = a; }
      void Print() const
      {  cout << "        X = " << X << endl
              << "  this->X = " << this->X << endl
              << "(*this).X = " << (*this).X << endl;
      }
   private:
      int X;
};
Test a(12);
a.Print();      // this = &a
```

```
        X = 12
  this->X = 12
(*this).X = 12
```

- Kiểu của this: `Test * const` (trong hàm const: `const Test * const`).
- **Bản chất:** lời gọi `a.Print()` được trình biên dịch dịch thành `Print(&a)`; trong hàm, mọi tên thành viên X được hiểu là `this->X`.
- Hàm static (mục 5) **không** có this.

4.2 Ứng dụng của this: gọi hàm nối tiếp

- Hàm set trả về **tham chiếu tới chính đối tượng** `return *this;` ⇒ **gọi nối tiếp** (cascading):

```
Time &Time::SetHour(int h)
{  Hour = (h >= 0 && h < 24) ? h : 0;
   return *this;           // trả về tham chiếu tới đối tượng
}
Time &Time::SetMinute(int m) { …; return *this; }
Time &Time::SetSecond(int s) { …; return *this; }
```

```
Time t;
t.SetHour(18).SetMinute(30).SetSecond(22);
t.PrintMilitary();         // 18:30
```

- `t.SetHour(18)` trả về **t** ⇒ tiếp tục `.SetMinute(30)` trên chính t. (Giống `cout << a << b`.)
- Nếu trả về `Time` (không có &) ⇒ SetMinute, SetSecond tác động lên **bản sao** ⇒ t chỉ được đặt giờ.
- **Ứng dụng thường gặp của this:**
  - Phân biệt thành viên với tham số **cùng tên**: `void SetX(int X) { this->X = X; }`.
  - Kiểm tra **tự gán** `if (this != &other)` (buổi 8).
  - Trả về `*this` để **gọi nối tiếp**; trả về `this` để lưu địa chỉ đối tượng vào danh sách.

5.1 Đối tượng được cấp phát động

- Dùng `new` để tạo đối tượng khi **chưa biết trước** số lượng hoặc cần đối tượng **sống lâu hơn** hàm tạo ra nó.

```
Time *p = new Time(8, 15);          // gọi constructor
p->PrintMilitary();
delete p;                           // gọi destructor
int n;  cin >> n;
Employee **ds = new Employee*[n];  // mảng n con trỏ
for (int i = 0; i < n; i++)
   ds[i] = new Employee(…);         // từng nhân viên
for (int i = 0; i < n; i++) delete ds[i];
delete[] ds;
```

- `new Time[n]` cần **constructor mặc định**; mảng con trỏ thì không (mỗi phần tử tạo riêng với tham số).

Ví dụ 6.3 – Theo dõi bằng thông báo

```
Employee *p = new Employee("Ann", "Le",
                           1, 2, 1990, 3, 4, 2015);
delete p;
```

```
Date constructor: 1/2/1990
Date constructor: 3/4/2015
Employee constructor: Ann Le
Employee destructor: Le, Ann
Date destructor: 3/4/2015
Date destructor: 1/2/1990
```

- Đối tượng thành viên (2 Date) cũng được tạo/hủy theo `new`/`delete` của đối tượng chứa.
- Quên `delete p` ⇒ **không** có 3 dòng destructor ⇒ rò rỉ.

- Danh sách liên kết đối tượng (mỗi nút là `new SinhVien(…)`), cây, đồ thị… đều dựa trên đối tượng cấp phát động.

Thành viên static

- **Thành viên dữ liệu static: một bản duy nhất** dùng chung cho **mọi** đối tượng của lớp (thuộc về lớp, không thuộc về đối tượng nào). Tồn tại ngay cả khi chưa có đối tượng nào. Phải **định nghĩa và khởi tạo một lần** ở ngoài lớp: `int Employee::Count = 0;`
- **Hàm thành viên static:** thuộc về lớp, gọi bằng `TenLop::Ham()`; **không** có con trỏ this ⇒ chỉ truy cập được thành viên **static**.

```
class Employee
{  public:
      Employee(…) : … { …; Count++; }   // tăng
      ~Employee()     { …; Count--; }   // giảm
      static int GetCount() { return Count; }
   private:
      …
      static int Count;                 // khai báo
};
int Employee::Count = 0;                // định nghĩa
```

- Bộ nhớ: mỗi Employee chứa FirstName, LastName, 2 Date; **Count nằm riêng**, không làm tăng sizeof(Employee).

Ví dụ 6.4 – Đếm số đối tượng đang tồn tại

```
cout << "Count = " << Employee::GetCount();      // 0
{  Employee e("Bob", "Jones", …);                // 1
   Employee *p = new Employee("Ann", "Le", …);   // 2
   delete p;                                      // 1
}                                                 // 0
cout << "Count = " << Employee::GetCount();
```

```
Count = 0  →  1  →  2  →  1  →  0
```

- Gọi được `Employee::GetCount()` **trước khi** có nhân viên nào.

5.3 Ví dụ static: mã tự tăng và lãi suất dùng chung

```
class TaiKhoan
{  public:
      TaiKhoan(double sd = 0)
      {  SoDu = sd;  MaTK = ++SoTKDaMo;  }   // mã tự tăng
      int GetMa() const { return MaTK; }
      double GetSoDu() const { return SoDu; }
      static void SetLaiSuat(double ls) { LaiSuat = ls; }
      void CongLai() { SoDu += SoDu * LaiSuat; }
   private:
      int MaTK;
      double SoDu;
      static int SoTKDaMo;       // dùng chung
      static double LaiSuat;     // dùng chung
};
int TaiKhoan::SoTKDaMo = 0;
double TaiKhoan::LaiSuat = 0.05;
int main()
{  TaiKhoan x(1000), y(500), z;
   cout << x.GetMa() << " " << y.GetMa() << " "
        << z.GetMa() << endl;              // 1 2 3
   TaiKhoan::SetLaiSuat(0.1);              // đổi cho MỌI TK
   x.CongLai();  y.CongLai();
   cout << x.GetSoDu() << " " << y.GetSoDu() << endl;
   cout << sizeof(TaiKhoan) << endl;
}
```

```
1 2 3
1100 550
16
```

- **Phân tích:**
  - `SoTKDaMo` tăng mỗi khi tạo tài khoản ⇒ mã 1, 2, 3 **không trùng**.
  - `SetLaiSuat(0.1)` gọi qua **tên lớp**, thay đổi **một chỗ** ⇒ mọi tài khoản cùng áp dụng lãi suất 10%.
  - `sizeof(TaiKhoan)` = 16 = int MaTK (4) + căn lề (4) + double SoDu (8): hai thành viên static **không** nằm trong đối tượng.
- **So sánh:**

Thành viên thường	Thành viên static
Mỗi đối tượng một bản	Cả lớp một bản
Truy cập qua đối tượng	Qua `TenLop::` hoặc đối tượng
Hàm có this	Hàm không có this
Khởi tạo trong constructor	Định nghĩa + khởi tạo ngoài lớp

6.1 Tóm tắt buổi 6

Khái niệm	Cú pháp	Ghi nhớ
Đối tượng hằng	`const Time t(…);`	Chỉ gọi được hàm const
Hàm thành viên const	`int Get() const;`	Không sửa dữ liệu; ghi const ở cả khai báo và định nghĩa
Thành viên const	`const int Step;`	Khởi tạo bằng danh sách khởi tạo
Đối tượng thành viên	`Date BirthDate;`	Tạo trước lớp chứa, hủy sau; khởi tạo bằng `: BirthDate(…)`
friend	`friend void F(A&);`	Truy cập private; không phải thành viên; dùng hạn chế
this	`this->X`, `return *this;`	Con trỏ tới đối tượng gọi hàm; gọi nối tiếp
Đối tượng động	`new Time(…)`, `delete p`	Gọi constructor/destructor
static	`static int Count;` + `int A::Count = 0;`	Dùng chung cho cả lớp; hàm static không có this

Câu hỏi nhanh

- Hàm static có thể là hàm const không? → **Không** (const áp dụng cho *this, mà hàm static không có this).
- Lớp A chứa đối tượng B và C (khai báo B trước C). Thứ tự constructor? → **B, C, A**; destructor: **A, C, B**.

6.2 Bài thực hành buổi 6 – Lớp NhanVien chứa Date, đếm đối tượng bằng static

Yêu cầu

- 1) Lớp **Date**: constructor có tham số mặc định và **kiểm tra hợp lệ** (tháng 1–12, ngày theo tháng, năm nhuận); `Print() const`; in thông báo trong constructor/destructor (để quan sát, có thể tắt bằng một biến static `HienThongBao`).
- 2) Lớp **NhanVien**: MaNV (tự tăng nhờ static `SoNVDaTao`), HoTen, `const Date NgaySinh`, `Date NgayVaoLam`, HeSoLuong; static `SoNVHienTai`; `static double LuongCoBan`.
  - Constructor dùng **danh sách khởi tạo** cho NgaySinh, NgayVaoLam.
  - Hàm const: `TinhLuong()`, `In()`; hàm set trả về `NhanVien &` để gọi nối tiếp: `nv.SetHoTen("An").SetHeSo(2.34);`.
  - Hàm static `GetSoNV()`, `SetLuongCoBan(double)`.
- 3) Hàm friend `int SoSanhThamNien(const NhanVien &a, const NhanVien &b)` truy cập trực tiếp NgayVaoLam.
- 4) main: tạo nhân viên cục bộ, mảng con trỏ cấp phát động; in số nhân viên sau mỗi thao tác; đổi lương cơ bản và in lại bảng lương; hủy toàn bộ.

Kiểm thử

Dự đoán trước thứ tự các dòng "Date constructor / NhanVien constructor / … destructor" rồi so với kết quả chạy; số nhân viên cuối chương trình phải về **0**.

6.3 Câu hỏi và bài tập

Câu hỏi

- 1) Vì sao nên khai báo const cho các hàm thành viên chỉ đọc? Đối tượng hằng gọi hàm không const thì sao?
- 2) Khi nào **bắt buộc** dùng danh sách khởi tạo thành viên?
- 3) Hàm friend khác hàm thành viên ở những điểm nào? Vì sao nói friend làm yếu tính đóng gói?
- 4) Con trỏ this là gì? Nêu 3 ứng dụng.
- 5) Phân biệt thành viên static và thành viên thường; vì sao hàm static không truy cập được thành viên thường?

Lỗi thường gặp với static

- Quên định nghĩa `int Employee::Count = 0;` ⇒ lỗi **liên kết**: `undefined reference to 'Employee::Count'`.
- Dùng `this` hoặc thành viên không static trong hàm static ⇒ lỗi biên dịch.
- Khởi tạo thành viên static trong constructor (`Count = 0;`) ⇒ mỗi lần tạo đối tượng lại **đặt lại** bộ đếm về 0.

Ứng dụng thường gặp của static

Đếm số đối tượng đang tồn tại; cấp mã tự tăng (`MaSV = ++SoSV`); hằng số chung của lớp (`static const double PI`); tham số cấu hình chung (lãi suất, lương cơ bản); hàm tiện ích không cần đối tượng (`PhanSo::UCLN(a, b)`).

6.4 Bài tập dự đoán kết quả

```
class A
{  public:
      A(int x) : v(x) { cout << "A" << v << " "; }
      ~A() { cout << "~A" << v << " "; }
   private:  int v;
};
class B
{  public:
      B() : a2(2), a1(1) { cout << "B "; n++; }
      ~B() { cout << "~B "; n--; }
      static int n;
   private:  A a1, a2;
};
int B::n = 0;
int main() { { B b; cout << B::n << " "; } cout << B::n; }
```

- **Đáp số:**

```
A1 A2 B 1 ~B ~A2 ~A1 0
```

- **Giải thích:**
  - Thành viên tạo theo **thứ tự khai báo** trong lớp (a1 rồi a2), **không** theo thứ tự trong danh sách khởi tạo (a2(2), a1(1)).
  - Sau khi các thành viên được tạo mới chạy thân constructor B ⇒ in "B", n = 1.
  - Ra khỏi khối: ~B trước, rồi hủy thành viên **ngược** thứ tự khai báo: ~A2, ~A1; n = 0.

Bài tập lập trình

- 1) Lớp `HocPhan` có static đếm số học phần, `const int SoTinChi`; lớp `SinhVien` chứa mảng con trỏ tới HocPhan đã đăng ký; hàm friend tính tổng tín chỉ.
- 2) Sửa lớp Time để mọi hàm set trả về `Time &` và viết một dòng lệnh đặt đủ giờ, phút, giây rồi in.